

GUARDRAILS DE SEGURANÇA - IA_AUTOMATE

Protocolos de Mitigação de Riscos e Controle de Execução Autônoma

06 de maio de 2026

1. Risco Crítico Identificado

O principal risco operacional identificado no estágio de prototipagem do sistema **IA_Automate** reside na capacidade da Inteligência Artificial de gerar e executar comandos destrutivos de forma não intencional ou mal interpretada. Este cenário ocorre tipicamente quando a IA recebe instruções ambíguas do usuário (ex: "limpe o ambiente para o novo módulo") e, na tentativa de otimizar o espaço ou organizar a estrutura, emite comandos que resultam na deleção de arquivos fonte essenciais ou diretórios de configuração.

Atenção: A probabilidade de ocorrência é classificada como **ALTA** na ausência de controles programáticos, dado que modelos de linguagem tendem a priorizar a conclusão da tarefa sobre a preservação da integridade do sistema de arquivos se não houver restrições explícitas.

Fator de Risco	Descrição	Nível de Impacto
Cenário	IA interpreta "limpar" como deleção física de arquivos .py, .json ou .ps1.	Crítico
Impacto	Perda total de progresso, corrupção do estado do projeto e interrupção do desenvolvimento.	Alto
Probabilidade	Alta (sem implementação de guardrails).	Moderado

2. Estratégia de Mitigação - 3 Camadas

Para garantir a segurança do ambiente de desenvolvimento, o sistema implementa uma arquitetura de defesa em profundidade composta por três camadas de proteção independentes e complementares.

2.1. Camada 1: Whitelist de Operações Perigosas

Esta camada atua no nível semântico do comando. O sistema possui uma lista exaustiva de operações permitidas. Qualquer comando que não conste explicitamente nesta lista é sumariamente rejeitado antes de atingir o interpretador de scripts.

- **Operações Bloqueadas por Padrão:**delete, remove, unlink, rm, rmdir.
- **Operações Permitidas:**create_file, edit_file, read_file, append_content, copy_file.
- **Implementação:**O JSON Schema valida o campo comando contra um enum restrito.

2.2. Camada 2: Whitelist de Caminhos (Allowed Paths)

Define o perímetro geográfico de atuação da IA. O sistema valida se o caminho (path) alvo de qualquer operação está contido dentro do diretório raiz do projeto.

- **Padrão Permitido:**/projeto_root/** (ex: /Users/dev/Desktop/Tools/Interface_Adapta/*).
- **Padrão Bloqueado:**Diretórios de sistema, pastas de outros usuários ou raízes de unidade (ex: /System32/, /etc/, C:/Windows/).
- **Validação:**O script de execução resolve o caminho absoluto e verifica a pertinência ao diretório autorizado.

2.3. Camada 3: Backup Automático Antes de Modificações

Atua como mecanismo de recuperação de desastres (Disaster Recovery). Antes de qualquer operação de escrita ou modificação (`edit\_file`), o sistema realiza uma cópia de sombra do estado atual do arquivo.

- **Local de Armazenamento:**Pasta Obsoleto/ (integrada à estrutura do bootstrap).
- **Versionamento:**Utilização de sufixos temporais ou sequenciais (ex: main_V01.bak).
- **Recuperação:**Em caso de erro lógico da IA, o sistema permite o rollback imediato para a versão estável anterior.

3. Implementação no JSON Schema

O controle de segurança é incorporado diretamente na estrutura de dados que a IA deve seguir. Abaixo, a representação de como o Schema impõe as camadas 1 e 2.

```
{
```

```
"seguranca": {<br/>
"permissions": ["file_read", "file_create", "file_edit"],<br/>
"allowed_paths": ["/Users/dev/Desktop/Tools/Interface_Adapta/"]
},
"comando": "delete_file",<br/>
"parametros": {<br/>
"arquivo": "app.py"
}
}
```

Resultado da Validação: REJEITADO. O comando `delete_file` não consta na lista de permissões autorizadas, disparando uma exceção de segurança imediata.

4. Fluxo de Validação

O fluxo abaixo descreve o ciclo de vida de um comando, desde a geração pela IA até a execução final no sistema host:

1. **Geração:** IA gera o comando em formato JSON e o disponibiliza no clipboard.
2. **Captura:** O sistema monitora o clipboard e captura o objeto JSON.
3. **Validação Camada 1 (Operação):** O sistema verifica se o comando pertence ao enum de operações seguras. *Falha:* Rejeita e notifica a IA.
4. **Validação Camada 2 (Path):** O sistema valida se o caminho alvo está dentro da whitelist de diretórios. *Falha:* Rejeita e notifica a IA.
5. **Ação Camada 3 (Backup):** Se o comando for de modificação, o sistema move a versão atual para a pasta Obsoleto/.
6. **Execução:** O comando é processado programaticamente (PowerShell/Node).
7. **Feedback:** O resultado (sucesso/erro) é retornado para a janela de comunicação da IA.

5. Checklist de Segurança

Todo comando gerado deve ser submetido ao seguinte checklist de pré-execução:

- O comando está explicitamente listado no enum de operações permitidas?

- O caminho (path) alvo está contido na whitelist de diretórios autorizados?
- Foi verificado se o arquivo alvo não é um arquivo de sistema protegido?
- O backup preventivo foi concluído com sucesso na pasta de obsoletos?
- O comando está livre de caracteres de escape ou injeção (;, |, &&)?
- O timeout de execução foi configurado para no máximo 30 segundos?
- A operação foi devidamente registrada nos logs de auditoria?

6. Resposta a Falhas e Aprendizado da IA

Quando um comando é bloqueado, o sistema não apenas interrompe a execução, mas fornece um feedback estruturado para que a IA possa corrigir seu comportamento autonomamente.

Exemplo 1: Tentativa de Deleção*Resposta do Sistema:*"Erro de Segurança: A operação 'delete_file' é restrita. Para remover conteúdo, utilize 'edit_file' enviando uma string vazia ou remova as linhas indesejadas mantendo a estrutura do arquivo."

Exemplo 2: Acesso Fora do Escopo
Resposta do Sistema: "Erro de Escopo: O caminho '/etc/hosts' não está na whitelist de permissões. A IA_Automate só possui autorização para modificar arquivos dentro de '/projeto root/'."

7. Recomendações Futuras

Para a evolução do projeto rumo à Fase 4 (Produção/Expansão), recomendam-se as seguintes implementações adicionais:

1. **Rate Limiting:** Implementar um teto máximo de comandos por minuto para evitar loops infinitos de escrita.
2. **Assinatura Digital:** Cada comando gerado pela IA deve ser acompanhado de um hash SHA-256 para garantir que o conteúdo não foi alterado entre o clipboard e a execução.
3. **Human-in-the-loop (HITL):** Para operações que envolvam múltiplos arquivos simultaneamente, exigir uma confirmação visual do usuário na interface.
4. **Rollback Atômico:** Implementar transações onde, se um grupo de comandos falhar, todos os arquivos retornam ao estado anterior ao início do lote.

[illegible]

ORIA DE SEGURANÇA

Local e data: Brasília, DF, 06 de maio de 2026

Documento gerado automaticamente pelo sistema de governança IA_Automate.